juliacon

Enhancing Deterministic Voice Control with Safe LLM Interaction

Samuel Omlin[1]

[1]Swiss National Supercomputing Centre (CSCS), ETH Zurich, Lugano, Switzerland

ABSTRACT

Small desktop tasks on personal computers remain insufficiently served by assistants with fixed command sets, tool-using artificial intelligence agents for which action approval is impractical in practice, and chat-only interaction that is too cumbersome for such tasks. This paper presents an approach that extends deterministic voice control with large language model interaction through fast command recognition, high-quality free-speech recognition, spoken output generation, clipboard- and selection-aware context access, predefined command layers, and composable primitives, while keeping system actions under deterministic command control. Implemented in JustSayIt.jl, the approach supports extremely concise primitive-based composition, enabling a minimalistic voice chatbot in three lines as well as application-specific command dictionaries such as focused Jupyter assistance, thereby demonstrating both expressive power and more efficient workflow-specific interaction. The resulting system enables seamless and safe integration of large language model intelligence into small everyday personal computer tasks, supports almost human-like follow-up through spoken responses, and bridges a practical gap in human-artificial-intelligence interaction.

Keywords

deterministic voice control, large language models, speech interfaces, programmable voice assistants, human-AI interaction, desktop automation

1. Introduction

Seamless and safe integration of large language model (LLM) intelligence into small, everyday tasks on a personal computer (PC) remains insufficiently served. One adjacent class is voice assistants whose functionality is exposed through predefined intents, sample utterances, and skill interfaces rather than programmable application-specific interaction tailored to a user's active workflow [1]. Another adjacent class is artificial intelligence (AI) agents and other LLM-based systems that act through tool use. Recent published work on autonomous agent safety emphasizes that such agents operate over personal information and device settings, can cause harmful side effects, and require explicit safety mechanisms, robust safeguards, and user trust [2, 3]. In that design space, per-action approval quickly becomes cumbersome, and the operational temptation is to slide toward unsafe yolo-style autonomy instead of keeping actions under deterministic control. A third adjacent class is plain chat-based interaction. Multitasking dialogue research treats switching between conversation and real-time tasks as a core difficulty, and chat-based conversational agents require explicit mechanisms for concurrent conversation tracking and memory even to manage context across interactions [4, 5]. The niche addressed here is therefore not a closed assistant capability catalog, not unrestricted tool autonomy, and not stand-alone chat, but deterministic voice control extended with LLM interaction for small desktop workflows, together with application-specific commands that make that interaction more efficient while keeping system actions explicit and safe.

This paper presents an approach for 1) integrating LLM interaction into a deterministic voice-control system so that LLM intelligence is available for small everyday tasks while system actions remain under safe deterministic command control; and 2) application-specific LLM interaction that is safe and programmable. The approach was successfully implemented in JustSayIt.jl.

2. Approach

The approach extends deterministic voice control with LLM interaction capabilities rather than starting from an unconstrained LLM interface. The deterministic foundation itself is not repeated here; it is the basis established in the 2024 JustSayIt paper [6] and the JuliaCon 2023 talk [7], and the present contribution adds a safe interaction layer on top of that foundation.

At the interaction level, the design uses a dual speech-to-text (STT) architecture: fast constrained recognition is used for commands, while high-quality free-speech recognition is used for LLM interaction and direct speech-to-text. Text-to-speech (TTS) integration provides spoken LLM output, enabling follow-up exchanges that approach almost human-like interaction. Clipboard and selection access are treated as idiomatic context sources, so LLM interaction can operate directly on the material currently being edited, inspected, or selected in the active application.

These LLM, STT, TTS, clipboard, and selection capabilities are exposed alongside the deterministic control mechanisms through four layers. First, a command submodule provides predefined generic LLM interaction commands; the application-specific example in Listing 2 shows how those commands are reused in a focused workflow. Second, parallel command submodules provide predefined generic STT, TTS, and clipboard or selection access commands. Third, a small set of idiomatic composable primitives provides base-language-like building blocks, similar in spirit to essential primitives such as `print`, so that new voice-plus-LLM commands can be composed directly without defining new functions; Table 1 and Listing 1 illustrate this layer in the next section. Fourth, lower-level LLM functionality remains available for maximum programmability, in particular for use in voice argument functions.

Table 1. : Composable primitives for LLM interaction, voice input, text capture, and typed or spoken output.

`listen`	Convert speech to text
`take`	Get clipboard content
`grab`	Get selected text
`ask`	Query LLM
`ask!`	Follow up on previous LLM query
`type`	Type text at cursor
`say`	Convert text to speech

3. Implementation and Results

The approach was successfully implemented in JustSayIt.jl[1]. The implementation integrates tightly with PromptingTools.jl [8] to enable sophisticated LLM interaction and uses Ollama [9] as additional option for local LLM execution; it uses Vosk [10] and faster-whisper [11] for the dual STT path, and supports multiple text-to-speech backends.

Table 1 gives the seven composable primitives for LLM interaction, voice-, clipboard-, and selection-based input, and typed or spoken output. Together they provide a compact root-level interface for defining LLM interaction directly in command dictionaries.

Listing 1 demonstrates the expressive power of this primitive layer. A minimal voice chatbot is defined entirely by composing speech input, LLM follow-up, and spoken output, showing how seamless and safe integration of LLM intelligence into small everyday PC tasks can be expressed with very little syntax.

Listing 2 demonstrates the complementary result for application-specific command dictionaries. Here, concise focused commands are defined by reusing predefined generic LLM commands rather than assembling the interaction from lower-level components, enabling more efficient LLM interaction in a concrete Jupyter workflow.

Listing 1: Minimal voice chatbot composed with the primitives in Table 1.

```
llm_commands = Dict("chatbot" => () ->  while true
                        listen() |> ask! |> say
                    end)
```

Listing 2: Application-specific Jupyter commands built from predefined LLM commands. In this example, `LLM.type_answer` answers an uttered instruction directly, whereas `LLM.type_text_answer_to` answers with respect to selected text or clipboard content; the first command types the appropriate IPython magic for a spoken task and the second types a markdown explanation of the selected cell, demonstrating concise application-specific LLM interaction.

```
jupyter_llm_commands = Dict(
  "generate magic" => () -> LLM.type_answer(instruction_prefix="
     Generate the appropriate IPython magic command for the
     following task using proper syntax (%line magic or %%cell
     magic) with necessary options:"),
  "explain cell" => () -> LLM.type_text_answer_to("Explain what
     the selected cell code does in simple terms. Include key
     functions, algorithms, and data transformations. Format as
     markdown with clear section headings."),
)
```

[1]Release target for the presented contribution: v0.4.0, `https://github.com/omlins/JustSayIt.jl/releases/tag/v0.4.0`.

4. Conclusions

The approach enables seamless and safe integration of LLM intelligence into small, everyday tasks on PC while keeping system actions under deterministic command control. Application-specific command dictionaries enable more efficient LLM interaction in focused workflows, and text-to-speech integration supports follow-up exchanges that approach almost human-like interaction. In this way, the approach addresses the niche defined in the introduction and bridges a practical gap in human-AI interaction.

5. References

[1] Nan Zhang, Xianghang Mi, Xuan Feng, XiaoFeng Wang, Yuan Tian, Feng Qian, et al. Dangerous skills: Understanding and mitigating security risks of voice-controlled third-party functions on virtual personal assistant systems. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 1381–1396, 2019. doi:10.1109/SP.2019.00016.

[2] Juyong Lee, Dongyoon Hahm, June Suk Choi, W. Bradley Knox, and Kimin Lee. Mobilesafetybench: Evaluating safety of autonomous agents in mobile device control. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(44):37565–37573, 2026. doi:10.1609/aaai.v40i44.41090.

[3] Jason Jabbour and Vijay Janapa Reddi. Generative AI agents in autonomous machines: A safety perspective. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, pages 1–13, 2025. doi:10.1145/3676536.3698390.

[4] Fan Yang, Peter A. Heeman, and Andrew Kun. Switching to real-time tasks in multi-tasking dialogue. In *Proceedings of the 22nd International Conference on Computational Linguistics - COLING '08*, volume 1, pages 1025–1032. Association for Computational Linguistics, 2008. doi:10.3115/1599081.1599210.

[5] Victor R. Martinez and James Kennedy. A multiparty chat-based dialogue system with concurrent conversation tracking and memory. In *Proceedings of the 2nd Conference on Conversational User Interfaces*, pages 1–9. ACM, 2020. doi:10.1145/3405755.3406121.

[6] Samuel Omlin. Justsayit.jl: A fresh approach to open source voice assistant development. *The Proceedings of the JuliaCon Conferences*, 6(66):121, 2024. doi:10.21105/jcon.00121.

[7] Samuel Omlin. Quick assembly of personalized voice assistants with justsayit. JuliaCon 2023 talk abstract and video, 2023. `https://pretalx.com/juliacon2023/talk/review/9MJFPDJV9DR7ANUXPSP9ZWJRFWSE83EY`.

[8] svilupp. Promptingtools.jl. Project documentation and README, 2025. `https://github.com/svilupp/PromptingTools.jl`.

[9] Ollama. Ollama. Project website, 2026. `https://ollama.com/`.

[10] Nickolay V. Shmyrev and contributors. Vosk speech recognition toolkit. Project website, 2020. `https://alphacephei.com/vosk/`.

[11] SYSTRAN. faster-whisper. Project documentation and README, 2025. `https://github.com/SYSTRAN/faster-whisper`.